

CS-484 Final Project

Jylen Tate

CS-484 Reinforcement Learning
Computer Science and Mathematics
jtate3@crimson.ua.edu

Abstract—This project focuses on training a robotic arm to perform a task-oriented manipulation skill using reinforcement learning. The assigned task was to press down on a jar using an affordance-based hand posture. A simulated robotic environment was used to allow the robot to interact with the jar and learn through repeated training trials. Proximal Policy Optimization (PPO) was applied as the reinforcement learning algorithm, and a custom reward function was designed to encourage proper hand positioning, stable contact, and downward pressing motion. The trained model was evaluated using success rates and examples of successful and failed trials. The results demonstrate how reinforcement learning can be used to teach a robot a specific object manipulation skill, while also showing challenges such as unstable contact, inaccurate positioning, and incomplete presses.

Index Terms—Reinforcement learning, robotic manipulation, affordance learning, Proximal Policy Optimization, task-oriented grasping

I. INTRODUCTION

Robotic manipulation is an important area in robotics because robots must be able to interact with objects in a way that matches the task they are trying to complete. A robot should not only recognize an object, but also understand how to use the object for a specific action. For example, in my case, the task of pressing down on a jar requires the robot to approach the object from above, establish stable contact, and apply controlled downward force to complete the task. While a simple task in nature, the robot must learn proper movement, contact, and control while avoiding unstable interactions such as applying uncontrolled downward force, missing the object, knocking the jar over, etc.

Using the provided simulation environment and base implementation, my approach focused on modifying and applying the existing reinforcement learning framework to the assigned jar-pressing task. The robot was trained in a simulated environment where it could repeatedly interact with the jar and improve its behavior through trial and error. A key part of my approach was adjusting the hand pose to better match the pressing affordance. For this task, a flatter and more open hand posture was more effective than a closed or grasping posture because the goal was not to pick up the jar, but to press down on it. Proximal Policy Optimization, or PPO, was used as the reinforcement learning algorithm because it had already shown success in the provided implementation for other affordance-based tasks. PPO was also useful because its clipping mechanism helps prevent drastic policy updates, allowing the new policy to stay close to the previous one while still improving over time. Moreover, the reward function was

a crucial part of my approach because it needed to reward the agent for getting close to the jar or within an acceptable error bound, without rewarding it to the degree that simply being near the object counted as completing the task. For this reason, I kept the two-stage approach. First, the robot needed to position itself above or near the top of the jar, and then it needed to apply controlled downward force to complete the pressing action.

The results showed that reinforcement learning can be used to teach a robot a specific task-oriented manipulation skill, but performance depends heavily on the reward design and the stability of the robot-object interaction. Successful trials occurred when the hand made proper contact with the jar and applied a clear downward motion. Failed trials were usually caused by poor positioning, unstable contact, or the robot applying too much or too little controlled force during the pressing motion. Ultimately, my approach demonstrated both the potential and the difficulty of using proximal policy optimization reinforcement learning for affordance-based robotic manipulation.

II. METHOD

A. Technical Description of Method

As mentioned previously, my approach used Proximal Policy Optimization (PPO) to train the robotic hand to complete the jar-pressing task. PPO is a reinforcement learning algorithm that updates a policy based on the rewards received from repeated interactions with the environment. In this project, the policy receives observations from the simulation and outputs continuous actions that control the movement of the robotic hand. The goal of training is to learn a policy that positions the hand above the jar and applies a controlled downward pressing motion. PPO was selected because it is designed to improve the policy while preventing the new policy from changing too drastically from the previous policy. This is useful for robotic manipulation because large policy updates can lead to unstable movements, poor contact, or unsafe interactions with the object. PPO controls this using a clipped objective function [1]:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip} \left(r_t(\theta), 1 - \epsilon, 1 + \epsilon \right) \hat{A}_t \right) \right] \quad (1)$$

where

$$r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \quad (2)$$

In these equations, $\pi_\theta(a_t | s_t)$ is the updated policy, $\pi_{\theta_{old}}(a_t | s_t)$ is the previous policy, s_t is the current state, a_t is the action selected by the policy, $\hat{A}_t$ is the estimated advantage, and ϵ is the clipping parameter. The ratio $r_t(\theta)$ measures how much the new policy differs from the old policy. The clipping term limits this difference, which helps PPO train more steadily.

Furthermore, the reward function was designed around two main parts of the pressing task. First, the robot needed to move its hand to the correct position above the jar. Second, the robot needed to make contact and apply downward force in a controlled way. The overall reward function was written as:

B. The Reward Function

$$R = \begin{cases} R_1, & \text{if the hand is not in the target position} \\ 80 + R_2, & \text{if the hand reaches the target position} \end{cases} \quad (3)$$

The first-stage reward, R_1 , encouraged the robot to move toward the target press position:

$$R_1 = \frac{1}{d+1} + 50D - C + 2(I_x + I_y + I_z) \quad (4)$$

where

$$d = \|p_h - p_t\|_2 \quad (5)$$

- p_h is the current hand or palm position.
- p_t is the target hand position above the jar.
- d is the distance between the hand and the target press position.
- $\frac{1}{d+1}$ is the distance reward. As the hand gets closer, d decreases and the reward increases.
- D is the direction reward, which measures whether the hand is moving in the correct direction toward the jar.
- C is the collision penalty based on the number of unwanted contact points before the press stage.
- I_x , I_y , and I_z are indicator values that equal 1 if the hand is close enough in the x , y , or z direction, and 0 otherwise.

The second-stage reward, R_2 , encouraged stable contact and pressing behavior:

$$R_2 = 2N_c + \frac{1}{F+1} + \frac{1}{\tau+1} \quad (6)$$

where

- N_c is the number of contact points between the robotic hand and the jar
- F is the total contact force imbalance
- τ is the total torque imbalance
- $2N_c$ rewards the robot for making contact with the jar
- $\frac{1}{F+1}$ and $\frac{1}{\tau+1}$ reward more stable contact, where lower force/torque imbalance gives a higher reward

The final reward also included bonus and penalty terms:

$$R_{\text{final}} = R + B \quad (7)$$

where

$$B = \begin{cases} +100000, & \text{if the pressing task succeeds} \\ +10000, & \text{if contact is within the friction cone} \\ -10000, & \text{if the episode times out} \\ -20000, & \text{if the object moves out of range} \end{cases} \quad (8)$$

C. Code Implementation

After defining the reward mathematically, the next step was implementing this logic inside the simulation environment. The function `_reward()` was responsible for deciding which stage reward should be used. If the hand was not yet inside the target position range, the environment used the first-stage reward, R_1 . Once the hand reached the target position, the environment changed to Stage 2 and used the second-stage reward, R_2 , with an additional constant reward of 80. This directly matches the piecewise reward function shown earlier:

```
if self.stage == 2:
    reward = 80 + reward_s2

if self.stage == 1:
    reward = reward_s1
```

$$R = \begin{cases} R_1, & \text{if the hand is not in the target position} \\ 80 + R_2, & \text{if the hand reaches the target position} \end{cases} \quad (9)$$

The condition for switching stages was based on the function `inPos(self.gl_error)`, which checks whether the hand is within the accepted error range in the x , y , and z directions.

The first-stage reward was implemented in `reward_s1()`. This stage encouraged the robotic hand to move toward the target position above the jar. The code first obtained the current palm position and calculated its distance from the target position.

```
p_hand = p.getLinkState(
    self._dual.dualUid,
    self.hp.dualEndEffectorIndex
)[4]

dist = self.hp.distant(p_hand, self.p_new)
reward_move = 1 / (dist + 1)
```

This corresponds to the distance reward term:

$$\frac{1}{d+1} \quad (10)$$

where `p_hand` represents p_h , `self.p_new` represents p_t , and `dist` represents d . As the hand moves closer to the target position, d decreases and the distance reward increases.

The direction reward was then calculated by comparing the desired direction of motion with the current direction of the hand.

```

initial_direction = np.array(self.p_new) \
    - np.array(self.hp.endEffectorPos)

current_direction = np.array(p_hand) \
    - np.array(self.hp.endEffectorPos)

reward_direct = self.hp.calculate_direction(
    initial_direction,
    current_direction
)

```

This implements the direction term D in the reward equation. The full reward multiplies this value by 50 so that moving in the correct direction gives stronger feedback to the agent.

The collision penalty was implemented by counting unwanted contact points between the hand and the jar before the pressing stage.

```

contact_points = p.getContactPoints(
    self._dual.dualUid,
    self.object
)

contact_num = len(contact_points)
reward_collision = contact_num if contact_num > 0
else 0

```

This corresponds to the collision penalty term C . If the robot makes unwanted contact before reaching the correct position, the reward is reduced.

The main Stage 1 reward was then calculated as:

```

reward = reward_move + 50 * reward_direct \
    - reward_collision

```

This matches the first part of the Stage 1 equation:

$$R_1 = \frac{1}{d+1} + 50D - C \quad (11)$$

Additional rewards were added when the hand was close enough in the x , y , and z directions.

```

if self.sl_x(self.near_error):
    reward += 2

if self.sl_y(self.near_error):
    reward += 2

if self.sl_z(self.near_error):
    reward += 2

```

These conditions represent the indicator terms I_x , I_y , and I_z . Each indicator adds 2 to the reward when the hand is within the acceptable error bound in that direction. Therefore, the complete Stage 1 reward is:

$$R_1 = \frac{1}{d+1} + 50D - C + 2(I_x + I_y + I_z) \quad (12)$$

Once the hand reached the final target range, the first-stage reward was set to 80.

```

if self.inPos(self.gl_error):
    reward = 80

```

This helped mark the completion of the positioning stage and allowed the task to transition into the pressing stage. The second-stage reward was implemented in `reward_s2()`. This stage rewarded the robot for making contact with the jar and maintaining stable force and torque behavior.

```

contact_points = p.getContactPoints(
    self._dual.dualUid,
    self.object
)

force, torque = self.check_equilibrium()

contact_num = len(contact_points) * 2 \
    + 1 / (sum(force) + 1) \
    + 1 / (sum(torque) + 1)

```

This corresponds to the second-stage reward equation:

$$R_2 = 2N_c + \frac{1}{F+1} + \frac{1}{\tau+1} \quad (13)$$

In the code, `len(contact_points)` represents N_c , the number of contact points between the robotic hand and the jar. The function `check_equilibrium()` calculates the force and torque values from the PyBullet contact information. The terms $\frac{1}{F+1}$ and $\frac{1}{\tau+1}$ reward stable contact because smaller force and torque imbalance values produce larger rewards.

After the stage reward was calculated, the function `step1()` added final bonus or penalty terms based on the result of the episode.

```

if self.fail_reason == "time_out":
    reward = reward - 10000

if self.fail_reason == "out_of_range":
    reward = reward - 20000

if self._graspSuccess:
    reward = reward + 100000

if self.in_friction_cone():
    reward = reward + 10000

```

This implements the final reward equation:

$$R_{\text{final}} = R + B \quad (14)$$

where B represents the final bonus or penalty. A large positive reward was added when the pressing task succeeded, and an additional reward was added when the contact forces were within the friction cone. Penalties were applied when the episode timed out or when the jar moved outside the valid range.

The source code also contained an additional object slip term from the original implementation:

```

if self.object_slip == 1:
    reward = reward + 40000

```

Together, these components formed the foundation of the learning process. PPO provided the policy update method, the reward function defined the desired behavior, and the simulation environment allowed the robot to repeatedly practice the jar-pressing task until a usable policy was learned.

III. RESULTS

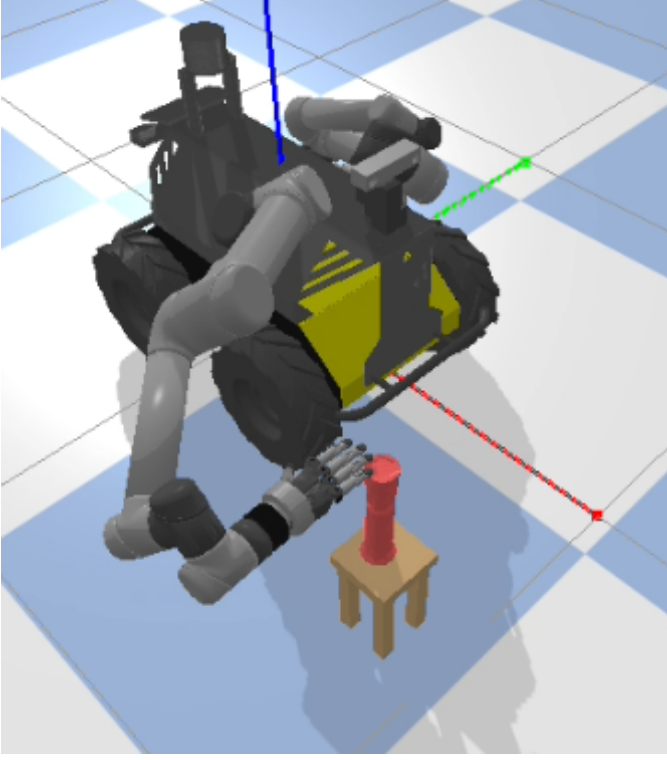


Fig. 1. Target affordance assigned for the project: pressing down on the jar.

Fig. 1 shows the assigned target affordance for this project. The task required the robotic hand to press down on the jar.

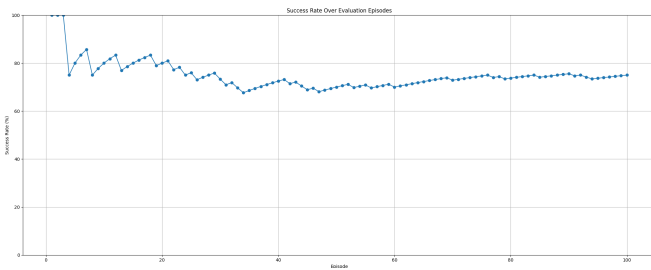


Fig. 2. Success rate over evaluation episodes for the trained PPO policy.

Fig. 2 shows the model's success rate over 100 evaluation episodes. After an initial drop, the success rate stabilized around the low-to-mid 70% range, showing that the policy learned the jar-pressing task but was not fully consistent across all trials.

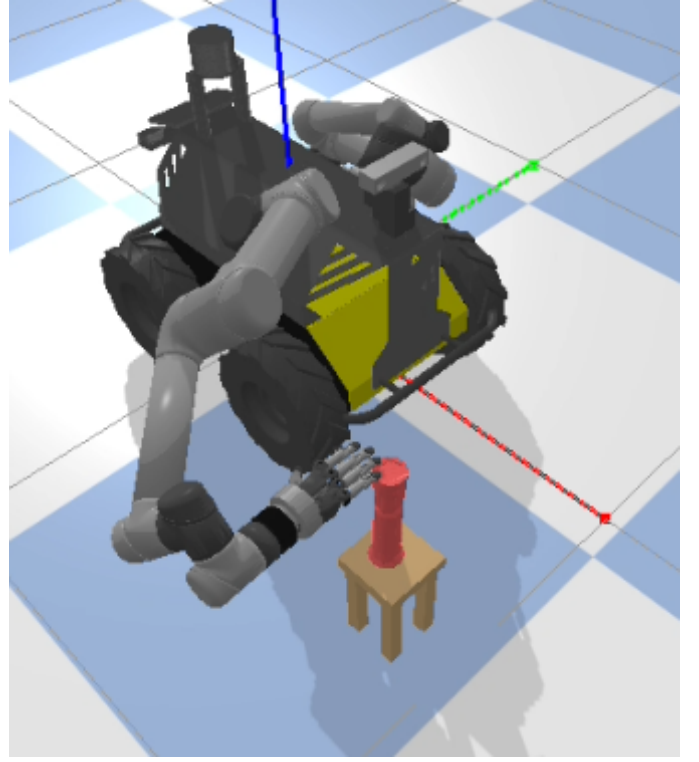


Fig. 3. Successful jar-pressing trial in the simulation environment.

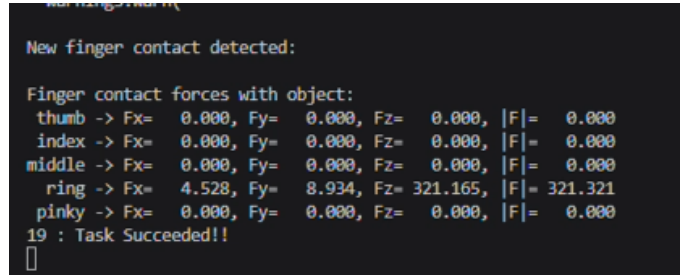


Fig. 4. Terminal output showing that the jar-pressing task succeeded.

Fig. 3 and Fig. 4 show a successful evaluation trial. The robotic hand reached the jar and made contact for the jar-pressing task.

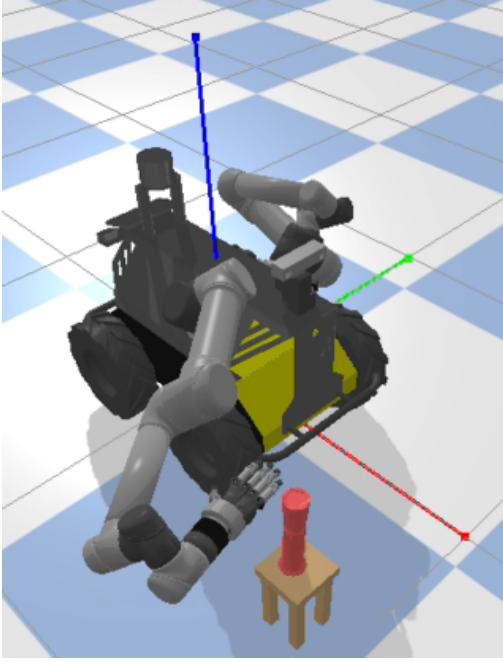


Fig. 5. Failed jar-pressing trial in the simulation environment.

```
19 : stop due to time out
Episode 4: reward=57662.202516555786, success=False, success rate=75.00%
```

Fig. 6. Terminal output showing the failed trial due to timeout.

Figure 5 and Figure 6 show an example of a failed jar-pressing trial in the simulation environment. In this run, the robotic hand was not able to complete the pressing motion before the episode reached the maximum time limit. Although the hand moved toward the jar, it did not reach a stable enough contact position to successfully apply the press. As a result, the trial ended in a timeout, as shown in Figure 6. This failure highlights one of the main limitations of the policy: small errors in positioning or contact timing could prevent the robot from completing the task successfully.

IV. DISCUSSION OF RESULTS

A. PPO Advantages and Disadvantages

Advantages:

- PPO is more stable than many basic policy-gradient methods because its clipping mechanism limits how much the policy can change during each update.
- PPO works well with continuous action spaces, which was useful because the robotic hand needed smooth movement to approach and press the jar.
- Compared to Monte Carlo methods, PPO can update the policy using shorter batches of experience instead of waiting for only complete episode returns, which is useful because robotic manipulation episodes can be long

and unstable. This allowed the agent to improve its jar-pressing behavior more efficiently during training.

Disadvantages:

- PPO does not automatically understand the goal of the task, so the task must be carefully shaped through rewards, success conditions, and penalties.
- PPO can struggle with precise contact tasks because small changes in hand position, force, or timing can determine whether the press succeeds or fails as seen while I was experimenting.
- Training can be inconsistent across trials. Even if the model learns the general pressing motion, it may still fail sometimes due to unstable contact, poor positioning, or timeout.

Ultimately, the approach was successful because the hand pose was adjusted to better match the pressing task, PPO's small, clipped policy updates helped the robot improve without making unstable changes in movement and the two-stage reward function approach allowed my policy to reach a 75% during the evaluation trial.

B. The Reward Function

As discussed previously, the designed reward function was structured around a two-stage approach because the robot could not successfully press the jar until it first reached the correct position. In the first stage, the reward focused on positioning the hand above the jar using key factors such as the distance reward, direction reward, collision penalty, and x, y, and z position indicators. These factors were important because the robot needed to move close to the correct target area, move in the proper direction, avoid unwanted contact too early, and stay within an acceptable error range before attempting the press. In the second stage, the reward focused on contact and pressing behavior using the number of contact points, force stability, and torque stability. These factors were important because simply reaching the jar was not enough, the robot also needed to make stable contact and apply controlled downward force without slipping, pushing incorrectly, or creating unstable motion. From observing my policy in action, I believe these factors worked fairly well because the robot was usually able to move toward the jar, reach the correct pressing area, and make contact with the object. The staged reward helped break the task into smaller goals, which made learning more manageable. However, one limitation is that the reward function mainly checks whether the hand reaches the correct position and makes contact with the jar. It does not fully measure how much downward force is applied during the press. Because of this, the robot learned the general pressing behavior, but the reward design could be improved by adding a more direct reward for downward motion or downward contact force on the jar. Ultimately, with a 75% during an evaluation trial I'd say the reward function worked well for guiding the robot toward the jar and completing the press task.

V. CONCLUSION

In conclusion, this project successfully trained a robotic hand to complete a jar-pressing manipulation task using Proximal Policy Optimization (PPO). The task setup was modified to better match the pressing affordance, including adjusting the hand pose so that the robot could apply pressure more effectively. A two-stage reward function was also designed, where the first stage guided the hand toward the correct position above the jar and the second stage rewarded stable contact and pressing behavior. The final trained policy learned the general pressing motion and achieved approximately a 75% success rate during evaluation.

The results showed that PPO was a strong choice for this task because its clipped policy updates allowed the robot to improve gradually without making unstable changes. The two-stage reward function also worked well because it divided the task into smaller learning goals, making it easier for the agent to first learn proper positioning and then stable contact. One important observation from this project was that positioning was one of the most critical factors in policy performance. Before reaching the final policy, several different training runs were tested and abandoned because the hand pose or reward structure did not guide the robot toward successful pressing behavior. For example, an earlier grasp-like hand pose was not well suited for this task because the closed hand made it difficult for the agent to press downward effectively.

Although the final model performed well overall, it was not perfect. Some trials still failed due to timeout, unstable contact, poor positioning, or incorrect pressing behavior. Another limitation was that the reward function did not directly measure downward pressing force. In some unsuccessful policies, the robot either pressed too hard and knocked the jar over or continued pressing without completing the task properly. I learned that future work could improve the reward design by adding a more direct term for downward motion or downward contact force. Overall, the final policy demonstrated that the chosen PPO approach, combined with the modified hand pose and two-stage reward function, was effective for learning the jar-pressing task.

REFERENCES

- [1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal Policy Optimization Algorithms,” arXiv:1707.06347, 2017.